

Data Structures Master Study Notes

In-Depth Technical Breakdown • Best vs. Worst Case Analysis • Time & Space Complexities

1. ArrayList / Dynamic Array

CONTIGUOUS MEMORY • INDEX-BASED ACCESS

OPERATION	BEST TIME	WORST TIME	WORST SPACE	EXACT TRIGGER CONDITION / MECHANIC
Access (by index)	$O(1)$	$O(1)$	$O(n)$	Direct pointer arithmetic: $base_address + (index * byte_size)$.
Search (by value)	$O(1)$	$O(n)$		Best: target is at index 0. Worst: target at index $n-1$ or missing (full linear scan).
Insertion	$O(1)$	$O(n)$		Best: appending at end with buffer capacity left. Worst: inserting at index 0 or buffer resize.
Deletion	$O(1)$	$O(n)$		Best: removing last item. Worst: deleting index 0 (shifts all $n-1$ elements left).

CORE MECHANICS

- Backed by a contiguous physical memory array.
- Excellent CPU cache locality due to sequential memory pre-fetching.
- Amortized Insertion:** Doubling array capacity on overflow keeps insertion average at $O(1)$.

SPACE & TRADEOFFS

- Worst-case Space $O(n)$:** Includes extra unused capacity padding (e.g., array capacity = 16 when size = 9).
- Costly shifts make middle insertions/deletions slow ($O(n)$).

2. Singly Linked List

NON-CONTIGUOUS • UNIDIRECTIONAL POINTERS

OPERATION	BEST TIME	WORST TIME	WORST SPACE	EXACT TRIGGER CONDITION / MECHANIC
Access / Search	$O(1)$	$O(n)$	$O(n)$	Best: accessing head node. Worst: target is at tail or index $n-1$ (sequential traversal).
Insertion	$O(1)$	$O(n)$		Best: insert at head (or tail if tail pointer exists). Worst: inserting at middle position without pointer.
Deletion	$O(1)$	$O(n)$		Best: remove head node. Worst: remove tail node (must search from head to find predecessor).

CORE MECHANICS

- Nodes distributed dynamically across memory heap.
- Each node holds: Data + Next Pointer.
- Inserting/deleting at head requires zero element shifting ($O(1)$).

SPACE & TRADEOFFS

- Tail Deletion Penalty:** Singly linked tail deletion is $O(n)$ because predecessor reference cannot be accessed backwards.
- Poor CPU cache performance due to pointer chasing across heap memory.

3. Doubly Linked List

BIDIRECTIONAL TRAVERSAL • NODE POINTERS

OPERATION	BEST TIME	WORST TIME	WORST SPACE	EXACT TRIGGER CONDITION / MECHANIC
Access / Search	$O(1)$	$O(n)$	$O(n)$	Best: head or tail node access. Worst: node in middle (can optimize from nearest end: $n/2$ steps).
Insertion	$O(1)$	$O(n)$		Best: insert at head/tail or given direct node pointer. Worst: search position first.
Deletion	$O(1)$	$O(n)$		Best: delete head/tail or given target node reference directly. Worst: delete by value without reference.

CORE MECHANICS

- Each node stores: Prev Pointer + Data + Next Pointer.
- Allows $O(1)$ removal of any node if its reference is already provided.
- Tail removal is $O(1)$ (unlike Singly Linked List).

SPACE & TRADEOFFS

- High Overhead:** 2 pointer references per item increases memory footprint significantly (e.g., 16 extra bytes/node on 64-bit systems).

4. Stack (LIFO) & Queue (FIFO)

ABSTRACT DATA TYPES • RESTRICTS ACCESS

STRUCTURE & OP	BEST TIME	WORST TIME	WORST SPACE	EXACT TRIGGER CONDITION / MECHANIC
Stack: Push / Pop	$O(1)$	$O(1)^*$	$O(n)$	Operates purely on top of stack pointer. (*Array-backed push amortized $O(1)$).
Stack: Search	$O(1)$	$O(n)$		
Queue: Enq / Deq	$O(1)$	$O(1)$		Enqueue at rear pointer, Dequeue at front pointer (LinkedList or Circular Array).
Queue: Search	$O(1)$	$O(n)$		Best: front item. Worst: last item at rear or missing.

5. Binary Trees (Unordered, BST, Self-Balancing)

HIERARCHICAL DATA • HEIGHT-BASED PERFORMANCE

TREE TYPE & OP	BEST TIME	WORST TIME	SPACE (WORST)	MECHANICS & DEGENERATE BEHAVIOR
Unordered Binary Tree (Search / Delete)	$O(1)$	$O(n)$	$O(n)$	No ordering. Requires complete BFS/DFS linear search across all n nodes.
Unbalanced BST (Search / Insert / Delete)	$O(1)$	$O(n)$	$O(n)$	Degenerate Tree: Inserting sorted data degrades tree into linked list ($h = n$).
Self-Balancing BST (AVL / Red-Black)	$O(1)$	$O(\log n)$	$O(n)$	Strict height bound ($h \approx \log_2 n$) enforced via continuous tree rotations ($O(1)$).

AUXILIARY STACK SPACE NUANCE

- Tree operations rely on recursive call stacks or explicit stacks/queues.
- Balanced Tree Stack Space:** $O(\log n)$ call stack frames.
- Skewed Tree Stack Space:** $O(n)$ recursive stack frames, causing potential stack overflow.

KEY TREE TERMS

- In-Order Predecessor/Successor:** Used during BST node deletion when a target node has 2 children.
- Rotations:** Low-level pointer changes in AVL/Red-Black trees to keep balance.